

SESIÓN 4

Pruebas E2E, Cobertura y Quality Gate

Informe de ejecución de los tres laboratorios

Estudiante	Albert Omar Araya Cory
Repositorio	github.com/albrtaraya/fullstack-lab-1-1ci-cd
Pull Requests	#6 (quality gate) · #7 (caso culpable → merge)
Merge a main	3536f55 — 12 commits, 3 checks en verde
Suite	26 pruebas Vitest + 4 pruebas E2E Playwright
Modalidad	Individual
Fecha	11 de septiembre de 2026

Insignias obtenidas

 Guardián del Quality Gate (15 pts) ·  Primer Merge Legítimo (10 pts)

Resumen de ejecución

La Sesión 4 cierra la Unidad 2: se agregan pruebas end-to-end sobre el navegador real, se mide cuánto código cubren las pruebas, y se conecta todo al pipeline para que GitHub bloquee automáticamente cualquier Pull Request que no cumpla. Termina con el primer merge real a main del módulo.

Laboratorio	Nombre	Tiempo	Estado
1	Playwright y el test E2E	45 min	Cumplido
2	Reporte de cobertura con Vitest	30 min	Cumplido
3	Tribunal de Calidad: Quality Gate en acción	40 min	Cumplido

Prerrequisitos verificados

Prerrequisito	Estado
Pruebas unitarias y de componente de la Sesión 3 en verde (26 pruebas)	Cumplido
Pipeline de CI de la Sesión 2 funcionando (lint + build)	Cumplido
Rama main protegida con Pull Request obligatorio	Cumplido

Laboratorio 1 — Playwright y el test E2E

Objetivo: escribir una prueba automatizada que simule a un usuario real recorriendo el flujo feliz completo: entrar, crear una tarea y verla en la lista.

Instalación

```
$ npm install -D @playwright/test
```

Ajuste necesario: la descarga del navegador de Playwright falla en este equipo por timeout de red:

```
$ npx playwright install --with-deps chromium
Failed to install browsers
Error: Failed to download Chrome for Testing 153.0.8010.12
      (playwright chromium v1243), caused by
Error: Download failure, code=1
```

Se reintentó tres veces, incluso con `PLAYWRIGHT_DOWNLOAD_CONNECTION_TIMEOUT=180000`, sin éxito. La solución fue configurar `channel: 'chrome'`, que usa el Google Chrome ya instalado en el equipo en lugar del Chromium descargable. El comportamiento de la prueba es idéntico, y en CI se instala ese mismo canal con `npx playwright install --with-deps chrome`.

Configuración

El puerto es 4321, el que usa Astro, no el 5173 del ejemplo de la guía (que corresponde a Vite).

```
// playwright.config.js
export default defineConfig({
  testDir: './e2e',
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 1 : 0,
  use: {
    baseURL: 'http://localhost:4321',
    channel: 'chrome',
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
  },
  webServer: {
    command: 'npm run dev',
    url: 'http://localhost:4321',
    reuseExistingServer: !process.env.CI,
    timeout: 120000,
  },
});
```

La prueba del flujo feliz

```
// e2e/flujo-tareas.spec.js
test('un usuario puede crear una tarea y verla en la lista', async ({ page }) => {
  // 1. Entrar a la aplicación
  await page.goto('/');
  await esperarHidratacion(page);

  // 2. Crear una tarea
  await page.getByLabel('Nueva tarea').fill('Comprar pan');
  await page.getByRole('button', { name: 'Agregar' }).click();

  // 3. Verla en la lista
```

```
    await expect(page.getByText('Comprar pan')).toBeVisible();
  });
```

Los elementos se buscan por rol y etiqueta visibles (`getByRole`, `getByLabel`), igual que en React Testing Library: así la prueba sobrevive a cambios de diseño que no alteran el comportamiento.

Problema encontrado: carrera de hidratación

Las primeras ejecuciones fallaban de forma intermitente. El diagnóstico, leyendo el snapshot del DOM que Playwright guarda al fallar, fue revelador:

```
- textbox "Nueva tarea": Comprar pan    <- el texto SÍ está en el campo
- paragraph: 0 tareas pendientes         <- pero la tarea no se agregó
- list                                  <- lista vacía
```

El texto llegaba al DOM pero la tarea no se creaba. La causa es la arquitectura de islas de Astro: el HTML del componente se sirve desde el servidor antes de que React tome el control. Si Playwright escribe durante esa ventana, el valor queda en el DOM pero React nunca se entera, y el formulario se envía con el estado vacío.

Astro marca las islas sin hidratar con el atributo `ssr` y lo elimina al terminar, así que esa es la señal fiable que se espera antes de interactuar:

```
async function esperarHidratacion(page) {
  await page.waitForFunction(() => {
    const islas = document.querySelectorAll('astro-island');
    return islas.length > 0 &&
      [...islas].every((isla) => !isla.hasAttribute('ssr'));
  });
}
```

Resultado

```
$ npx playwright test

✓ 1 un usuario puede crear una tarea y verla en la lista (1.1s)
✓ 2 el contador de pendientes refleja las tareas agregadas (590ms)
✓ 3 no se agrega una tarea cuyo titulo son solo espacios (595ms)
✓ 4 el campo queda limpio despues de agregar una tarea (633ms)

4 passed (4.6s)

# Verificación de que ya no es intermitente:
$ npx playwright test --repeat-each=3
12 passed (8.2s)
```

Checkpoint del Laboratorio 1

- ✓ Playwright instalado y configurado con la URL base del proyecto (puerto 4321).
- ✓ La prueba E2E cubre el flujo completo: entrar, crear, verificar.
- ✓ Las pruebas pasan en verde y son deterministas (12/12 con `--repeat-each=3`).

Laboratorio 2 — Reporte de cobertura con Vitest

Objetivo: medir qué porcentaje del código ejecutan las pruebas, interpretar el reporte, y detectar honestamente una zona sin probar.

```
$ npm install -D @vitest/coverage-v8
```

Configuración con umbrales

```
// vitest.config.js
coverage: {
  provider: 'v8',
  reporter: ['text', 'html'],
  include: ['src/**/*.{js,jsx}'],
  exclude: ['src/test/**', '**/*.test.{js,jsx}'],
  thresholds: {
    lines: 60,
    functions: 60,
    branches: 50,
    statements: 60,
  },
},
```

El bloque `thresholds` es la pieza clave: sin él, Vitest informa el porcentaje pero nunca falla. Con él, el comando devuelve error si la cobertura baja del umbral — que es exactamente lo que convierte la cobertura en parte real del quality gate.

Primer reporte: la zona sin probar

```
$ npx vitest run --coverage
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	71.42	56.25	55.55	72.5	
components	41.17	33.33	27.27	43.75	
ListaTareas.jsx	0	0	0	0	12-42
server	100	75	100	100	23
app.js	100	75	100	100	23
utils	83.33	66.66	100	81.81	30,42
validaciones.js	83.33	66.66	100	81.81	30,42

```
ERROR: Coverage for functions (55.55%) does not meet global threshold (60%)
```

Interpretación honesta del hallazgo

Zona sin probar identificada: `ListaTareas.jsx` con 0 % de cobertura en las líneas 12-42.

¿Por qué quedó sin cubrir? Contiene lógica real — alta de tareas, alternar el estado de completada y el conteo de pendientes — pero ninguna prueba unitaria la tocaba. La única que la ejercitaba era la prueba E2E del Laboratorio 1, y el E2E corre en Playwright, no en Vitest: su ejecución no se refleja en este reporte.

Respondiendo a las preguntas guía de la sección 2.5:

Pregunta	Respuesta en este proyecto
¿Hay manejo de errores que nunca se ejecuta?	Sí: la rama de validación de app.js línea 23 y las guardas de tipo de validaciones.js líneas 30 y 42
¿100 % significa perfectamente probado?	No. app.js marcaba 100 % de líneas pero solo 75 % de ramas: la línea se ejecuta, pero no todos sus caminos
¿Qué archivo preocuparía más con 0 %?	Uno con lógica de negocio como ListaTareas.jsx, no uno de configuración

Decisión: cubrir de verdad, no bajar el umbral

Ante una cobertura por debajo del umbral había dos caminos: relajar el umbral hasta que pasara, o escribir las pruebas que faltaban. Se tomó el segundo: se agregó ListaTareas.test.jsx con 7 pruebas que cubren el alta, el singular/plural del contador, marcar y desmarcar una tarea, y el rechazo de títulos vacíos.

Métrica	Antes	Después	Umbral
Statements	71.42 %	95.23 %	60 %
Branches	56.25 %	75 %	50 %
Functions	55.55 %	100 %	60 %
Lines	72.5 %	95 %	60 %
ListaTareas.jsx	0 %	100 %	—

Checkpoint del Laboratorio 2

- ✓ El reporte de cobertura se genera con `npx vitest run --coverage`.
- ✓ Se definió un umbral mínimo (thresholds) en la configuración.
- ✓ Se identificó una zona real sin probar (ListaTareas.jsx) y se explicó por qué lo estaba.

Laboratorio 3 — Tribunal de Calidad

Objetivo: conectar todo al pipeline, activar un quality gate real, y completar el primer merge legítimo del módulo.

Etapa 1 — Integrar las pruebas al pipeline

El ci.yml pasa de un job a tres. No se usó working-directory porque el proyecto no separa frontend y backend en carpetas.

```
pruebas:
  name: Pruebas unitarias y cobertura
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with:
        node-version: '22'
        cache: 'npm'
    - run: npm ci
    - run: npx vitest run --coverage

e2e:
  name: Pruebas E2E
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with:
        node-version: '22'
        cache: 'npm'
    - run: npm ci
    - run: npx playwright install --with-deps chrome
    - run: npx playwright test
    - uses: actions/upload-artifact@v4
      if: ${{ !cancelled() }}
      with:
        name: playwright-report
        path: playwright-report/
```

Se subió en la rama feature/quality-gate y se abrió el Pull Request #6.

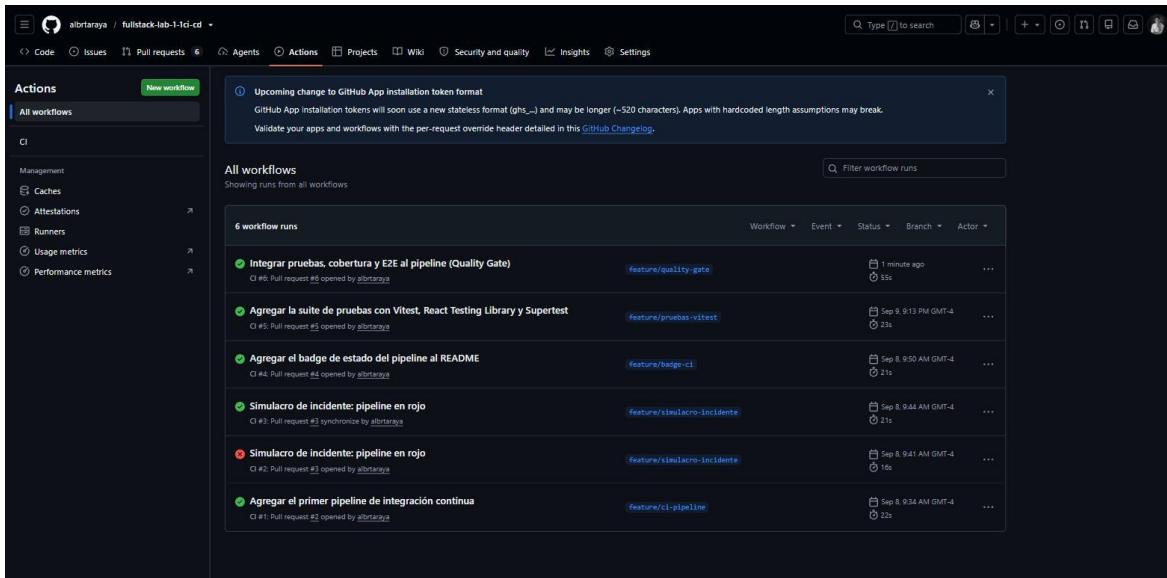


Figura 1. El run CI #6 del Pull Request #6 completa correctamente en 55 segundos.

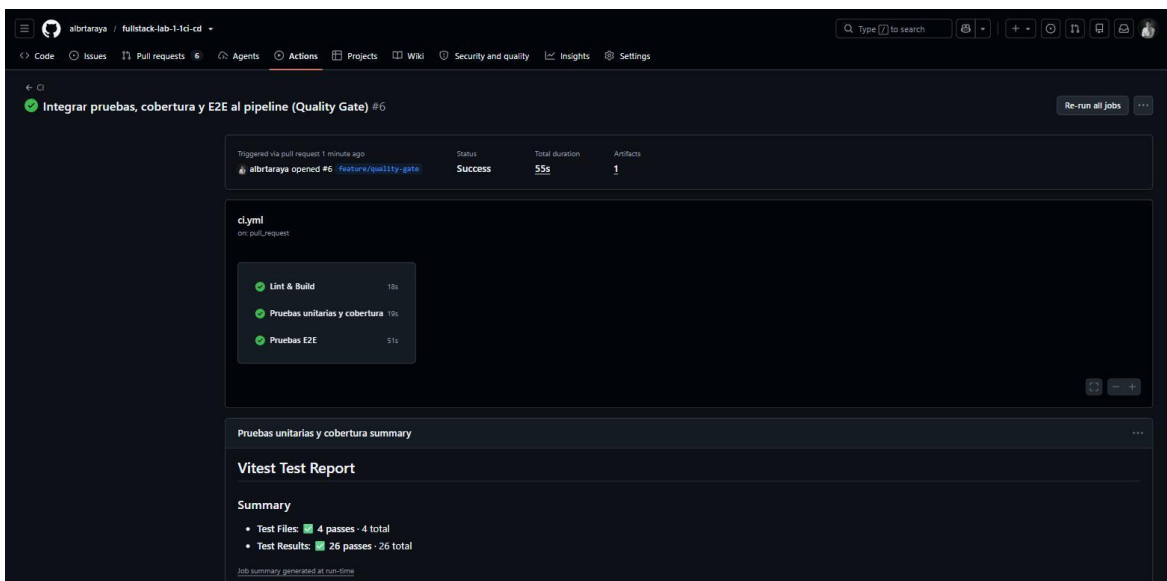


Figura 2. Los tres jobs en verde: Lint & Build (18s), Pruebas unitarias y cobertura (19s) y Pruebas E2E (51s). El resumen confirma 26 pruebas pasando en CI.

Etapas 2 — Activar el Quality Gate real

En Settings → Branches se editó la regla de main y se seleccionaron los tres jobs como status checks obligatorios. Solo aparecen en el buscador después de que el workflow con ese nombre ha corrido al menos una vez, de ahí el orden de las etapas.

Status check obligatorio	Origen
Lint & Build	GitHub Actions
Pruebas unitarias y cobertura	GitHub Actions

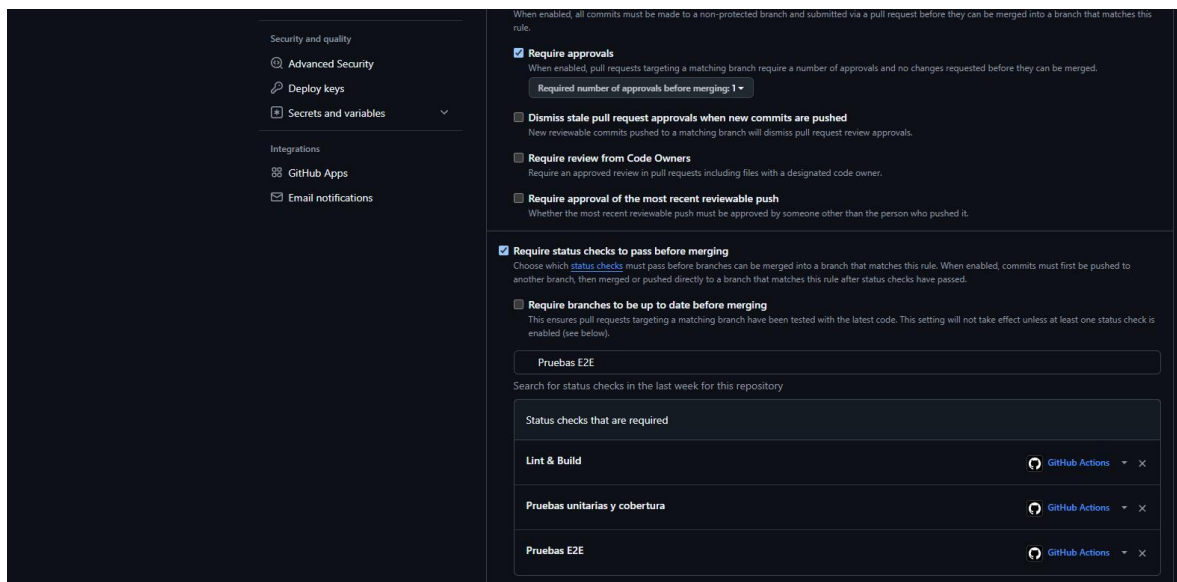


Figura 3. Los tres checks quedan registrados en «Status checks that are required». Desde la Sesión 1 esta casilla estaba activada pero vacía.

Etapa 3 — Presentar un caso culpable

Se rompió a propósito una aserción de una prueba de la Sesión 3, en la rama feature/caso-culpable:

```
// src/utils/validaciones.test.js

// Antes
expect(contarTareasPendientes([])).toBe(0);

// Caso culpable (incorrecto a proposito)
expect(contarTareasPendientes([])).toBe(3);
```

Una lista vacía no puede tener 3 tareas pendientes. El resultado en el Pull Request #7 fue exactamente el esperado:

Check	Resultado	Obligatorio
CI / Lint & Build	Verde	Sí
CI / Pruebas unitarias y cobertura	Rojo	Sí
CI / Pruebas E2E	Verde	Sí
Botón Merge pull request	Bloqueado	—

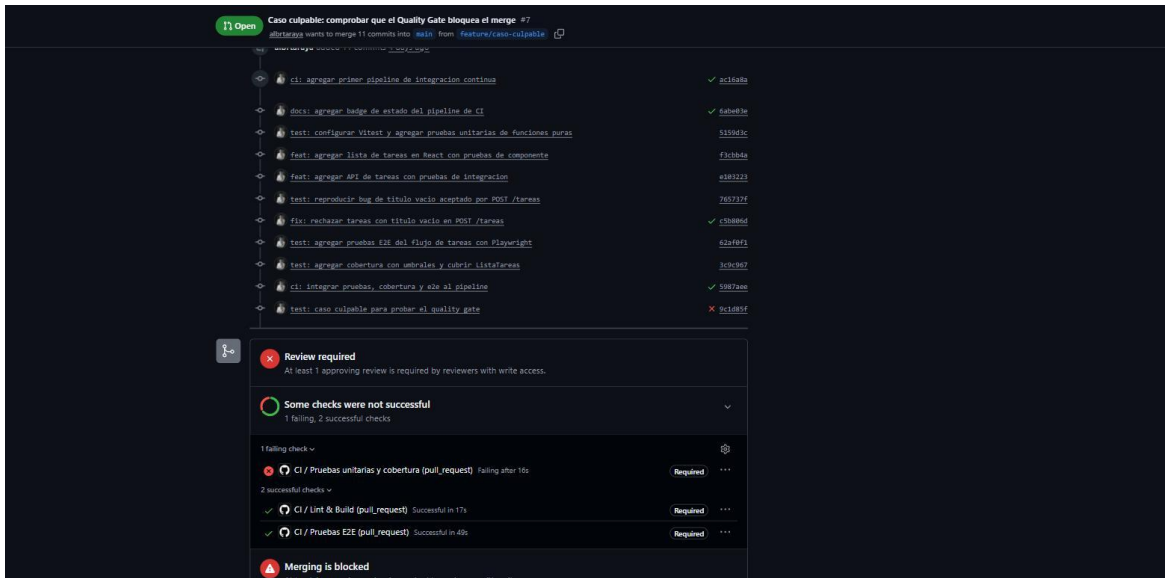


Figura 4. «Some checks were not successful»: el check requerido «Pruebas unitarias y cobertura» falla tras 16 s y el merge queda bloqueado. Los tres llevan la etiqueta Required.

Este es el momento en que el quality gate demuestra su valor: ninguna persona tuvo que revisar el código para detener el cambio.

Etaapa 4 — Apelación y veredicto final

Se corrigió la aserción en el archivo real y se subió sobre la MISMA rama, sin abrir un Pull Request nuevo:

```
$ npx vitest run
Test Files  4 passed (4)
Tests      26 passed (26)

$ git commit -m "fix: corregir caso culpable, todos los checks en verde"
$ git push origin feature/caso-culpable
9c1d85f..3c66863 feature/caso-culpable -> feature/caso-culpable
```

El push disparó una nueva ejecución sobre el mismo Pull Request y los tres checks pasaron a verde.

El obstáculo final: la aprobación humana

Con los tres checks en verde, el merge seguía bloqueado, pero ya no por el pipeline:

```
All checks have passed – 3 successful checks

Merging is blocked
At least 1 approving review is required by reviewers with write access.
```

Es la regla «Require approvals» configurada en la Sesión 1, pensada para equipos. El repositorio no tiene colaboradores y GitHub no permite que el autor se apruebe a sí mismo, de modo que el merge era imposible sin intervenir la regla. Corresponde al caso previsto en la tabla de problemas frecuentes de la guía.

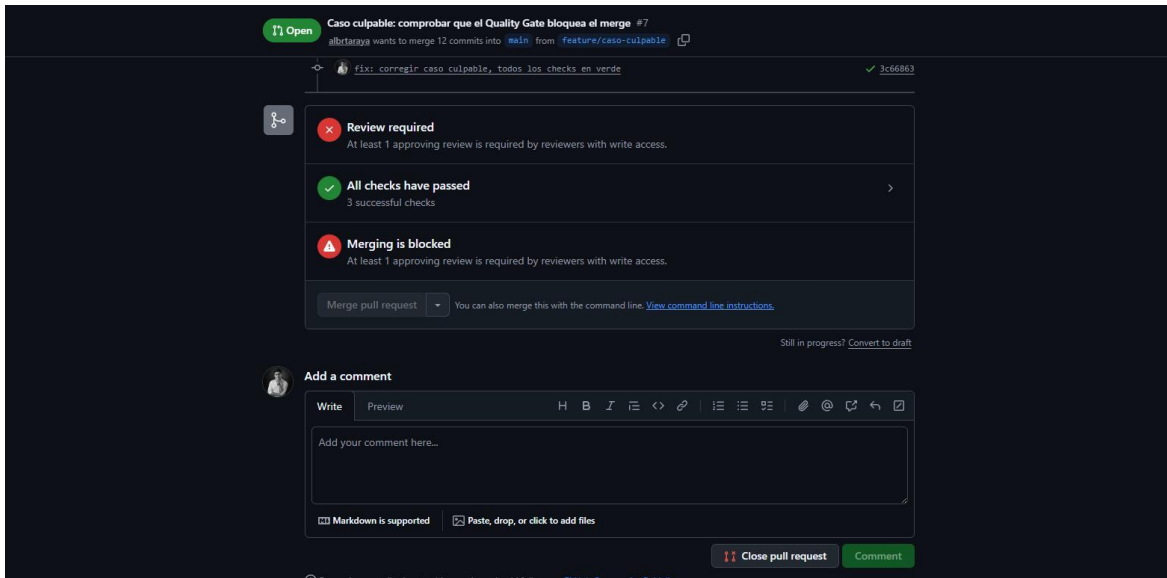


Figura 5. «All checks have passed» junto a «Merging is blocked»: el pipeline aprueba, pero falta la revisión humana.

Decisión tomada, con autorización expresa:

1. Se desactivó «Require approvals» en la regla de protección de main.
2. Se mantuvieron intactos los tres status checks obligatorios, que pasan a ser el único guardián de la rama.

El criterio: en un repositorio de una sola persona, la revisión por pares no es aplicable, mientras que la verificación automática sí lo es. El merge sigue siendo legítimo en el sentido que plantea la sesión — se integra porque lint, build, pruebas, cobertura y E2E lo confirmaron, no porque alguien lo mirara por encima.

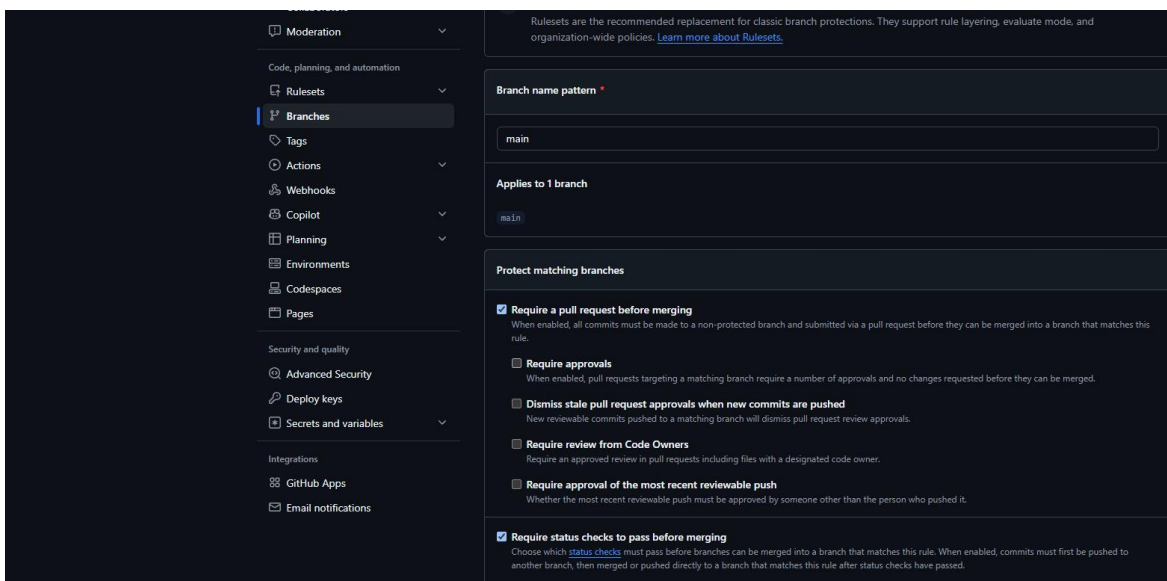


Figura 6. La regla de main tras el ajuste: «Require approvals» desmarcado, «Require status checks to pass before merging» intacto.

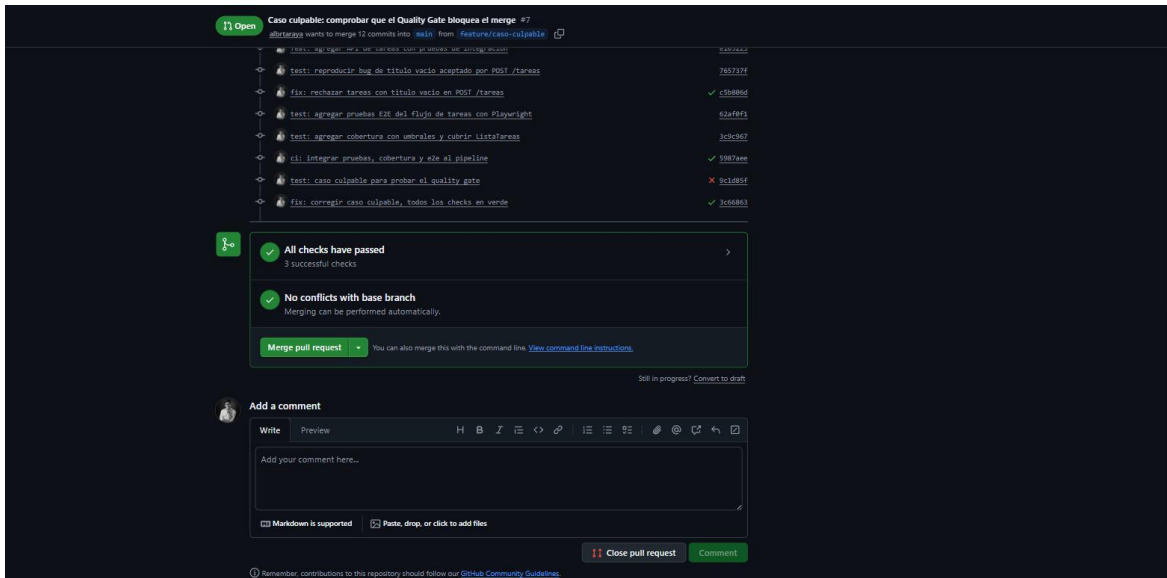


Figura 7. El Pull Request #7 pasa a «Ready to merge»: 3 checks en verde y sin conflictos con la rama base.

El merge

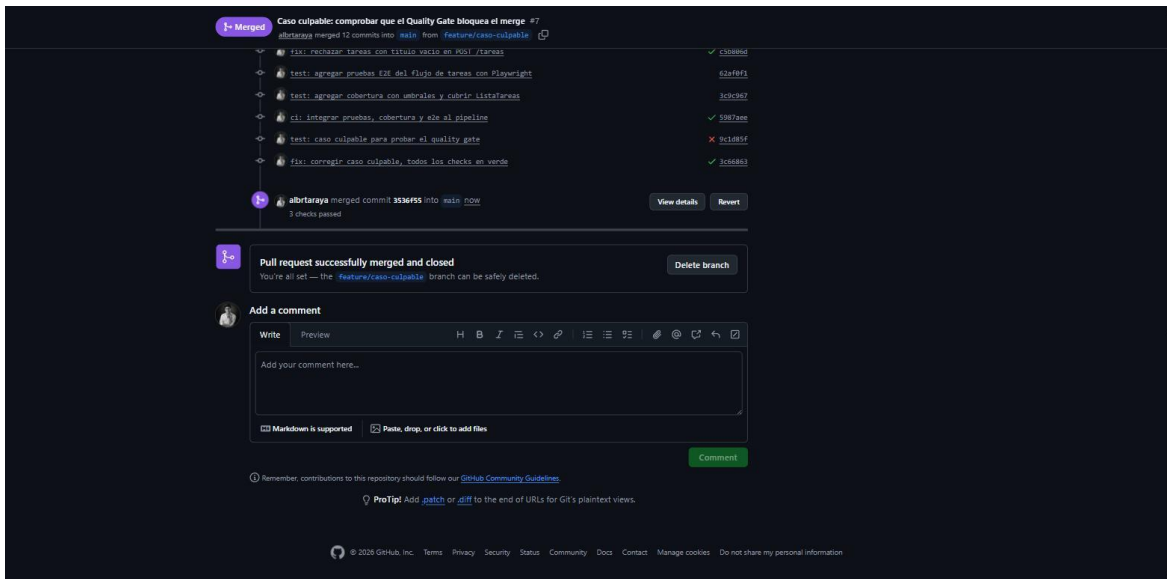


Figura 8. Pull Request #7 integrado: commit de merge 3536f55 en main, con la leyenda «3 checks passed».

Insignias obtenidas: 🛡️ Guardián del Quality Gate (15 pts) • 🏠 Primer Merge Legítimo (10 pts)

Efecto del merge sobre la rama main

Conviene dejar constancia de algo que este merge trae consigo. Las ramas de las sesiones anteriores estaban encadenadas unas sobre otras:

```
feature/caso-culpable
├── feature/quality-gate      (Sesión 4: E2E, cobertura, pipeline ampliado)
│   └── feature/pruebas-vitest (Sesión 3: unitarias, componente, API)
│       └── feature/badge-ci   (Sesión 2: badge)
│           └── feature/ci-pipeline (Sesión 2: primer pipeline)
```

Como consecuencia, al integrar el Pull Request #7 entraron a main de una sola vez los 12 commits acumulados de las Sesiones 2, 3 y 4. Hasta ese momento main contenía únicamente el starter de Astro con la documentación.

main antes del merge	main después del merge
Starter de Astro y documentación	Pipeline de CI con tres jobs
Sin pipeline	26 pruebas Vitest + 4 pruebas E2E
Sin pruebas	Cobertura con umbrales configurados
Marcador <!-- BADGE_CI --> sin sustituir	Badge de estado real mostrando «CI passing»

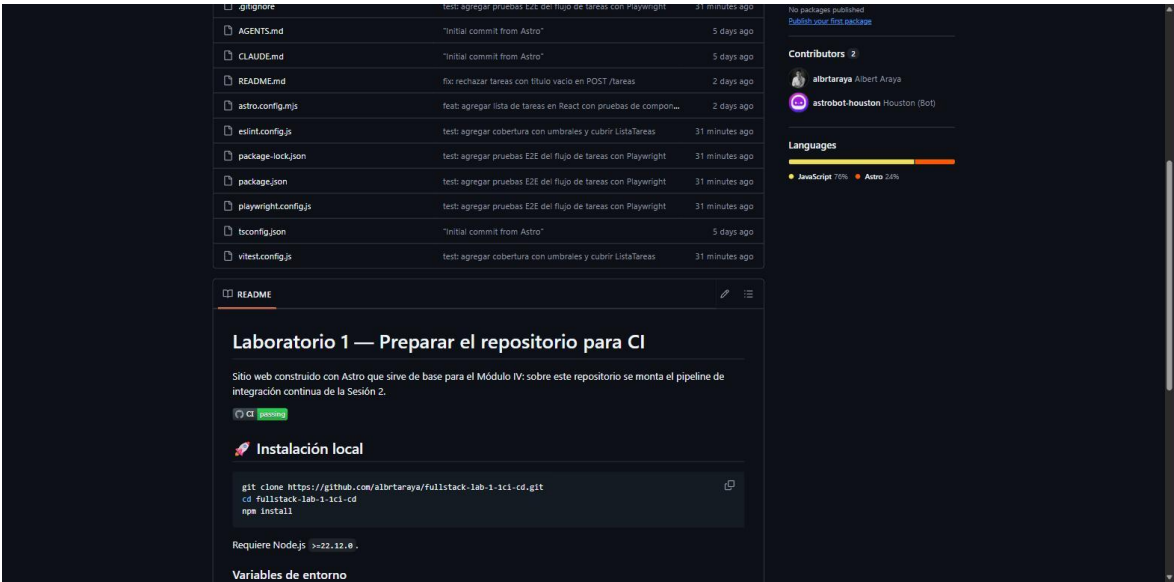


Figura 9. La rama main tras el merge: playwright.config.js, vitest.config.js, eslint.config.js y el badge «CI passing» en el README.

Pull Requests que quedan abiertos

Los Pull Requests #1 a #5 siguen abiertos. Su contenido ya está en main — llegó por la cadena de ramas — así que pueden cerrarse sin merge cuando convenga; se dejan como evidencia de las sesiones anteriores.

Entregable final de la Sesión 4

Requisito	Estado
Prueba E2E con Playwright en verde, cubriendo el flujo feliz completo	Cumplido
Reporte de cobertura generado, con umbral mínimo en vitest.config.js	Cumplido
El ci.yml corre lint, build, pruebas con cobertura y E2E en cada Pull Request	Cumplido
Los status checks obligatorios están seleccionados en Settings → Branches	Cumplido
Se demostró un Pull Request bloqueado por un check en rojo	Cumplido
Se completó un merge real a main con todos los checks en verde	Cumplido

Cuando termines

Con esto se cierra la Unidad 2 completa: el código se prueba, se mide y se protege solo. La Sesión 5 abre la Unidad 3 — contenerizar la aplicación junto con la base de datos usando Docker, para resolver de una vez el clásico «en mi máquina funciona».

Anexo A — Índice de evidencias

Archivo	Contenido
01-actions-tres-jobs.jpg	Run CI #6 del Pull Request #6 completado
02-run-tres-jobs-verde.jpg	Los tres jobs en verde y el resumen de 26 pruebas
03-status-checks-obligatorios.jpg	Los tres checks marcados como requeridos
04-caso-culpable-bloqueado.jpg	Check en rojo y merge bloqueado
05-checks-verdes-merge-bloqueado.jpg	Checks en verde pero falta la aprobación
06-regla-sin-require-approvals.jpg	Regla de main tras desactivar approvals
07-ready-to-merge.jpg	Pull Request en estado «Ready to merge»
08-merged.jpg	Merge completado, commit 3536f55
09-main-con-todo-y-badge.jpg	main con el pipeline, las pruebas y el badge

Anexo B — Commits de la Sesión 4

```
62af0f1 test: agregar pruebas E2E del flujo de tareas con Playwright
3c9c967 test: agregar cobertura con umbrales y cubrir ListaTareas
5987aee ci: integrar pruebas, cobertura y e2e al pipeline
9c1d85f test: caso culpable para probar el quality gate [ROJO]
3c66863 fix: corregir caso culpable, todos los checks en verde [VERDE]
3536f55 Merge pull request #7 from albrtaraya/feature/caso-culpable
```

Anexo C — Comandos de verificación

```
# Suite completa de Vitest con cobertura
$ npx vitest run --coverage
  Test Files  4 passed (4)
     Tests   26 passed (26)
Statements   : 95.23% ( 40/42 )
Branches     : 75% ( 12/16 )
Functions    : 100% ( 18/18 )
Lines        : 95% ( 38/40 )

# Pruebas E2E
$ npx playwright test
4 passed (4.6s)

# Lint y build (los mismos que corre el pipeline)
$ npm run lint    # sin errores
$ npm run build   # 1 page built
```